



Eigenvector Centrality

✓ [Directed](#) ?

✓ [Undirected](#) ?

(✓) [Heterogeneous nodes](#) ?

(✓) [Heterogeneous relationships](#) ?

✓ [Weighted relationships](#) ?

Introduction

Eigenvector Centrality is an algorithm that measures the **transitive** influence of nodes. Relationships originating from high-scoring nodes contribute more to the score of a node than connections from low-scoring nodes. A high eigenvector score means that a node is connected to many nodes who themselves have high scores.

The algorithm computes the eigenvector associated with the largest absolute eigenvalue. To compute that eigenvalue, the algorithm applies the [power iteration](#) ↗ approach. Within each iteration, the centrality score for each node is derived from the scores of its incoming neighbors. In the power iteration method, the eigenvector is L2-normalized after each iteration, leading to normalized results by default.

The [PageRank](#) algorithm is a variant of Eigenvector Centrality with an additional jump probability.

Considerations

There are some things to be aware of when using the Eigenvector centrality algorithm:

- Centrality scores for nodes with no incoming relationships will converge to 0 .
- Due to missing degree normalization, high-degree nodes have a very strong influence on their neighbors' score.



AI search

Syntax

This section covers the syntax used to execute the Eigenvector Centrality algorithm in each of its execution modes. We are describing the named graph variant of the syntax. To learn more about general syntax variants, see [Syntax overview](#).

Eigenvector Centrality syntax per mode

[Stream mode](#)[Stats mode](#)[Mutate mode](#)[Write mode](#)

Run Eigenvector Centrality in stream mode on a named graph.

CYPHER 

```
CALL gds.eigenvector.stream(  
  graphName: String,  
  configuration: Map  
)  
YIELD  
  nodeId: Integer,  
  score: Float
```

Table 1. Parameters

Name	Type	Default	Optional	Description
graphName	String	n/a	no	The name of a graph stored in the catalog.
configuration	Map	{}	yes	Configuration for algorithm-specifics and/or graph filtering.

Table 2. Configuration

Name	Type	Default	Optional	Description
<u>nodeLabels</u>	List of String	['*']	yes	Filter the named graph using the given node labels. Nodes with any of the given labels will be included.
<u>relationshipTypes</u>	List of String	['*']	yes	Filter the named graph using the given relationship types. Relationships with any of the given types will be included.
<u>concurrency</u>	Integer	4 ^[1]	yes	The number of concurrent threads used for running the algorithm.
<u>jobId</u>	String	Generated internally	yes	An ID that can be provided to more easily track the algorithm's progress.
<u>logProgress</u>	Boolean	true	yes	If disabled the progress percentage will not be logged.
<u>maxIterations</u>	Integer	20	yes	The maximum number of iterations of Eigenvector Centrality to run.
<u>tolerance</u>	Float	0.0000001	yes	Minimum change in scores between iterations. If all scores change less than the tolerance value the result is considered stable and the algorithm returns.
<u>relationshipWeightProperty</u>	String	null	yes	Name of the relationship property to use as weights. If unspecified, the algorithm runs unweighted.
sourceNodes	List or Node or Number	[]	yes	The nodes or node ids to use for computing Personalized Eigenvector Centrality.
scaler	String or Map	None	yes	The name of the scaler applied for the final scores. Supported values are <code>None</code> , <code>MinMax</code> , <code>Max</code> , <code>Mean</code> , <code>Log</code> , and <code>StdScore</code> . To apply scaler-specific configuration, use the Map syntax: <code>{scaler: 'name', ...}</code> .

1. In a GDS Session, the default is the number of available processors.

Table 3. Results

Name	Type	Description
nodeId	Integer	Node ID.
score	Float	Eigenvector score.

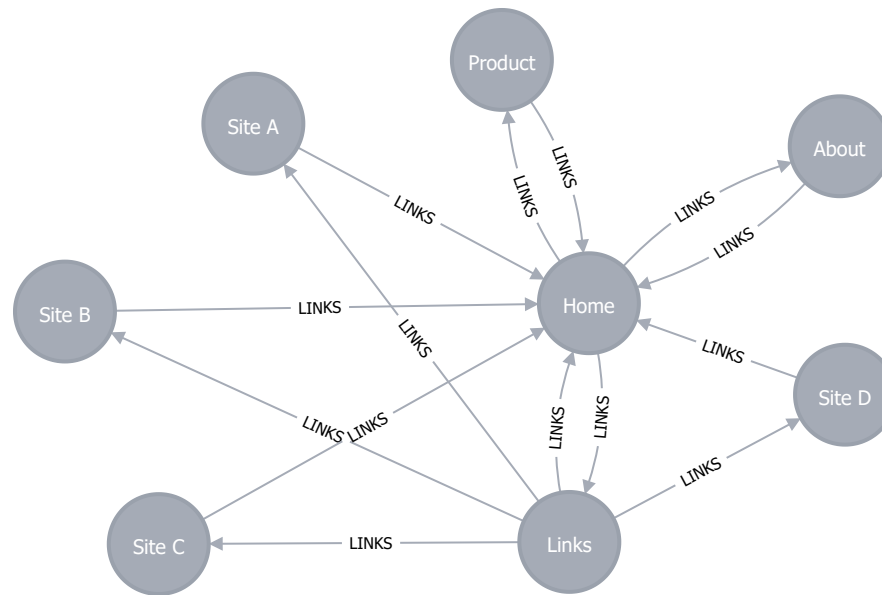
Examples

NOTE

All the examples below should be run in an empty database.

The examples use Cypher projections as the norm. Native projections will be deprecated in a future release.

In this section we will show examples of running the Eigenvector Centrality algorithm on a concrete graph. The intention is to illustrate what the results look like and to provide a guide in how to make use of the algorithm in a real setting. We will do this on a small web network graph of a handful nodes connected in a particular pattern. The example graph looks like this:



The following Cypher statement will create the example graph in the Neo4j database:

CYPHER

CREATE

```

(home:Page {name:'Home'}),
(about:Page {name:'About'}),
(product:Page {name:'Product'}),
(links:Page {name:'Links'}),
(a:Page {name:'Site A'}),
(b:Page {name:'Site B'}),
(c:Page {name:'Site C'}),
(d:Page {name:'Site D'}),

(home)-[:LINKS {weight: 0.2}]->(about),
(home)-[:LINKS {weight: 0.2}]->(links),
(home)-[:LINKS {weight: 0.6}]->(product),
(about)-[:LINKS {weight: 1.0}]->(home),
(product)-[:LINKS {weight: 1.0}]->(home),

```

[View all \(9 more lines\)](#)

This graph represents eight pages, linking to one another. Each relationship has a property called `weight`, which describes the importance of the relationship.

The following statement will project a graph using a Cypher projection and store it in the graph catalog under the name 'myGraph'.

CYPHER 

```
MATCH (source:Page)-[r:LINKS]->(target:Page)
RETURN gds.graph.project(
  'myGraph',
  source,
  target,
  { relationshipProperties: r { .weight } }
)
```

Memory Estimation

First off, we will estimate the cost of running the algorithm using the `estimate` procedure. This can be done with any execution mode. We will use the `write` mode in this example. Estimating the algorithm is useful to understand the memory impact that running the algorithm on your graph will have. When you later actually run the algorithm in one of the execution modes the system will perform an estimation. If the estimation shows that there is a very high probability of the execution going over its memory limitations, the execution is prohibited. To read more about this, see [Automatic estimation and execution blocking](#).

For more details on `estimate` in general, see [Memory Estimation](#).

The following will estimate the memory requirements for running the algorithm:

CYPHER 

```
CALL gds.eigenvector.write.estimate('myGraph', {
  writeProperty: 'centrality',
  maxIterations: 20
})
```

```
})  
YIELD nodeCount, relationshipCount, bytesMin, bytesMax, requiredMemory
```

Table 13. Results

nodeCount	relationshipCount	bytesMin	bytesMax	requiredMemory
8	14	696	696	"696 Bytes"

Stream

In the `stream` execution mode, the algorithm returns the score for each node. This allows us to inspect the results directly or post-process them in Cypher without any side effects. For example, we can order the results to find the nodes with the highest Eigenvector score.

For more details on the `stream` mode in general, see [Stream](#).

The following will run the algorithm in **stream** mode:

CYPHER 

```
CALL gds.eigenvector.stream('myGraph')  
YIELD nodeId, score  
RETURN gds.util.asNode(nodeId).name AS name, score  
ORDER BY score DESC, name ASC
```

Table 14. Results

name	score
"Home"	0.7465574981728249

name	score
"About"	0.33997520529777137
"Links"	0.33997520529777137
"Product"	0.33997520529777137
"Site A"	0.15484062876886298
"Site B"	0.15484062876886298
"Site C"	0.15484062876886298
"Site D"	0.15484062876886298

The above query is running the algorithm in `stream` mode as `unweighted`. Below, one can find an example for [weighted graphs](#).

Stats

In the `stats` execution mode, the algorithm returns a single row containing a summary of the algorithm result. For example Eigenvector stats returns centrality histogram which can be used to monitor the distribution of centrality scores across all computed nodes. This execution mode does not have any side effects. It can be useful for evaluating algorithm performance by inspecting the `computeMillis` return item. In the examples below we will omit returning the timings. The full signature of the procedure can be found in [the syntax section](#).

For more details on the `stats` mode in general, see [Stats](#).

The following will run the algorithm and return statistics about the centrality scores.

CYPHER 

```
CALL gds.eigenvector.stats('myGraph', {
  maxIterations: 20
})
YIELD centralityDistribution
RETURN centralityDistribution.max AS max
```

Table 15. Results

max
0.7465591431

Mutate

The `mutate` execution mode extends the `stats` mode with an important side effect: updating the named graph with a new node property containing the score for that node. The name of the new property is specified using the mandatory configuration parameter `mutateProperty`. The result is a single summary row, similar to `stats`, but with some additional metrics. The `mutate` mode is especially useful when multiple algorithms are used in conjunction.

For more details on the `mutate` mode in general, see [Mutate](#).

The following will run the algorithm in **mutate** mode:

CYPHER 

```
CALL gds.eigenvector.mutate('myGraph', {
  maxIterations: 20,
  mutateProperty: 'centrality'
})
YIELD nodePropertiesWritten, ranIterations
```

Table 16. Results

nodePropertiesWritten	ranIterations
8	20

Write

The `write` execution mode extends the `stats` mode with an important side effect: writing the score for each node as a property to the Neo4j database. The name of the new property is specified using the mandatory configuration parameter `writeProperty`. The result is a single summary row, similar to `stats`, but with some additional metrics. The `write` mode enables directly persisting the results to the database.

For more details on the `write` mode in general, see [Write](#).

The following will run the algorithm in **write** mode:

CYPHER 

```
CALL gds.eigenvector.write('myGraph', {  
  maxIterations: 20,  
  writeProperty: 'centrality'  
})  
YIELD nodePropertiesWritten, ranIterations
```

Table 17. Results

nodePropertiesWritten	ranIterations
8	20

Weighted

By default, the algorithm considers the relationships of the graph to be unweighted. To change this behaviour, we can use the `relationshipWeightProperty` configuration parameter. If the parameter is set, the associated property value is used as relationship weight. In the `weighted` case, the previous score of a node sent to its neighbors is multiplied by the normalized relationship weight. Note, that negative relationship weights are ignored during the computation.

In the following example, we use the `weight` property of the input graph as relationship weight property.

The following will run the algorithm in **stream** mode using relationship weights:

CYPHER 

```
CALL gds.eigenvector.stream('myGraph', {
  maxIterations: 20,
  relationshipWeightProperty: 'weight'
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 18. Results

name	score
"Home"	0.8328163407319487
"Product"	0.5004775834976313
"About"	0.1668258611658771
"Links"	0.1668258611658771

name	score
"Site A"	0.008327591469710233
"Site B"	0.008327591469710233
"Site C"	0.008327591469710233
"Site D"	0.008327591469710233

As in the unweighted example, the "Home" node has the highest score. In contrast, the "Product" now has the second highest instead of the fourth highest score.

NOTE

We are using `stream` mode to illustrate running the algorithm as `weighted`, however, all the algorithm modes support the `relationshipWeightProperty` configuration parameter.

Tolerance

The `tolerance` configuration parameter denotes the minimum change in scores between iterations. If all scores change less than the configured tolerance, the iteration is aborted and considered converged. Note, that setting a higher tolerance leads to earlier convergence, but also to less accurate centrality scores.

The following will run the algorithm in **stream** mode using a high **tolerance** value:

CYPHER 

```
CALL gds.eigenvector.stream('myGraph', {
  maxIterations: 20,
  tolerance: 0.1
})
YIELD nodeId, score
```

```
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 19. Results

name	score
"Home"	0.7108273818583551
"About"	0.3719400001993262
"Links"	0.3719400001993262
"Product"	0.3719400001993262
"Site A"	0.14116155811301126
"Site B"	0.14116155811301126
"Site C"	0.14116155811301126
"Site D"	0.14116155811301126

We are using `tolerance: 0.1`, which leads to slightly different results compared to the [stream example](#). However, the computation converges after three iterations, and we can already observe a trend in the resulting scores.

Personalised Eigenvector Centrality

Personalized Eigenvector Centrality is a variation of Eigenvector Centrality which is biased towards a set of `sourceNodes`. By default, the power iteration starts with the same value for all nodes: $1 / |V|$. For a given set of source nodes `S`, the initial value of each source node is set to $1 / |S|$ and to 0 for all remaining nodes.

The following examples show how to run Eigenvector centrality centered around 'Site A'.

The following will run the algorithm and stream results:

CYPHER 

```
MATCH (siteA:Page {name: 'Site A'}), (siteB:Page {name: 'Site B'})
CALL gds.eigenvector.stream('myGraph', {
  maxIterations: 20,
  sourceNodes: [siteA, siteB]
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 20. Results

name	score
"Home"	0.7465645391567868
"About"	0.33997203172449453
"Links"	0.33997203172449453
"Product"	0.33997203172449453
"Site A"	0.15483736775159632
"Site B"	0.15483736775159632
"Site C"	0.15483736775159632
"Site D"	0.15483736775159632

Scaling centrality scores

Internally, centrality scores are scaled after each iteration using L2 normalization. As a consequence, the final values are already normalized. This behavior cannot be changed as it is part of the power iteration method.

However, to normalize the final scores as part of the algorithm execution, one can use the `scaler` configuration parameter. A description of all available scalers can be found in the documentation for the [scaleProperties](#) procedure.

The following will run the algorithm in **stream** mode and returns normalized results:

CYPHER 

```
CALL gds.eigenvector.stream('myGraph', {
  scaler: "MINMAX"
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 21. Results

name	score
"Home"	1.0
"About"	0.312876962110942
"Links"	0.312876962110942
"Product"	0.312876962110942
"Site A"	0.0
"Site B"	0.0

name	score
"Site C"	0.0
"Site D"	0.0

Comparing the results with the [stream example](#), we can see that the relative order of scores is the same.

[Prev](#)[◀ Degree Centrality](#)[Next](#)[PageRank ▶](#)

Contents

- Introduction
- Considerations
- Syntax
- Examples
 - Memory Estimation
 - Stream
 - Stats
 - Mutate
 - Write
 - Weighted
 - Tolerance
 - Personalised Eigenvector Centrality

Scaling centrality scores

Graph Data Science courses on GraphAcademy

Follow our Graph Data Analytics learning path to learn how to apply graph thinking to your machine learning pipelines.

Enroll now

LEARN

 [Sandbox](#)
 [Neo4j Community Site](#)

SOCIAL

 [Twitter](#)
 [Meetups](#)
 [Github](#)
 [Stack Overflow](#)

CONTACT US →

US: 1-855-636-4532
Sweden +46 171 480 113

 [Neo4j Developer](#)[Want to Speak?](#)

UK: +44 20 3868

[Blog](#)

3223

[Neo4j Videos](#)

France: +33 (0) 1 88

[GraphAcademy](#)

46 13 20

[Neo4j Labs](#)

© 2026 Neo4j, Inc.

[Terms](#) | [Privacy](#) | [Sitemap](#)

Neo4j[®], Neo Technology[®], Cypher[®], Neo4j[®] Bloom[™] and Neo4j[®] Aura[™] are registered trademarks of Neo4j, Inc. All other marks are owned by their respective companies.